

AUDITORIA DE CÓDIGO • 2026

Auditoria de Segurança e Validação das Correções



Auditoria verificável de isolamento de dados, autorização, IDOR, segredos e XSS, acompanhada da remediação validada dos achados.

AUDITORIA-BASE 02 de setembro de 2026 • bb164eb957eaf688a4f7a5322750c6b49e5e7df8

REMEDIÇÃO 02 de setembro de 2026 • incluída no commit que contém este relatório

STATUS 2 achados corrigidos • 0 achados abertos

ESCOPO API, web, Prisma, Docker/Compose, Nginx, documentação, bundle e histórico Git alcançável

NOTA METODOLÓGICA

As cinco categorias solicitadas foram adaptadas à stack detectada. O ExtraOK não usa Supabase/RLS nem RBAC: o tenant é o usuário autenticado, e o isolamento é aplicado por `ownerId` nas queries Prisma. Todos os 23 handlers foram percorridos; gates do React foram cruzados com a API; segredos foram buscados no estado atual, nos cinco commits, em objetos Git adicionais e no bundle; sinks XSS foram pesquisados no frontend e backend.

01 • RESUMO EXECUTIVO

Dois achados corrigidos; nenhum risco confirmado permanece aberto

CRÍTICA ABERTA 0

ALTA ABERTA 0

MÉDIA CORRIGIDA 1

BAIXA CORRIGIDA 1

Severidade histórica dos corrigidos



Achados históricos por categoria

Banco sem tranca	0
Permissão no browser	0
IDOR	0
Chaves/configuração	2
XSS	0

Remediação concluída

NODE_ENV e DATABASE_URL agora falham de forma fechada quando ausentes. Os fallbacks previsíveis foram removidos sem alterar o endurecimento do cookie ou o limite do Studio.

Autorização preservada

Não foi encontrado acesso horizontal: listas, detalhes, mutações, agregações e o fluxo público vinculam o recurso ao usuário ou ao `jobId` derivado do token.

Avaliação pós-correção: os dois riscos de configuração identificados na auditoria-base foram corrigidos e cobertos por regressões. Testes focados, suíte completa, integração real, typecheck, lint e build foram aprovados; não restou achado confirmado aberto nas cinco categorias revisadas.

02 • STACK E MAPEAMENTO

Como as categorias foram traduzidas para o ExtraOK

Linguagem/runtime	TypeScript 6, ESM, Node.js 24.19.x	Backend	Fastify 5.12
ORM/query	Prisma 7.10 + adapter-pg	Banco	PostgreSQL 17
Autenticação	Sessão opaca persistida; cookie HttpOnly	Tenant	Um usuário = um tenant; filtro manual ownerId
Frontend	React 19, Vite 8, Router, Query, Zod	Deploy	Docker multi-stage, Compose e Nginx sem privilégio
Ausente	Supabase/RLS, papéis/RBAC, CI, Helm, Terraform e Kubernetes		

1

Banco sem tranca

Como não há RLS, a revisão seguiu o identificador autenticado até cada `findMany`, `findFirst`, `count`, `aggregate`, `update` e `delete`. Extras e links herdam posse por `Job`.

2

Permissão definida no navegador

Não existe `role/admin/can*`. O gate de autenticação e cada gate de estado da UI foram cruzados com as rotas e regras equivalentes no servidor.

3

IDOR

Todos os IDs em path, query ou body foram rastreados. Recursos privados exigem dono; a decisão pública cruza `extraId` com o `jobId` do link.

4

Chaves expostas e defaults

Foram varridos source, configs, exemplos, docs, Docker, todos os commits/refs, objetos Git extras e o bundle. O `.env` local foi comparado sem divulgar valores.

5

Inputs sem tratamento / XSS

Busca por sinks DOM/HTML, markdown, execução dinâmica, URLs controladas e HTML no backend; revisão manual dos locais que renderizam dados da API.

03 • RESULTADOS POR CATEGORIA

Cobertura, histórico e status da remediação

1	Banco sem tranca Isolamento manual por ownerId presente em todas as queries de negócio.	Sem achado
2	Permissão no navegador Não há papéis/RBAC; gates de estado têm validação equivalente na API.	Não se aplica
3	IDOR Todos os handlers por ID validam dono ou vínculo ao token.	Sem achado
4	Chaves/configuração Dois achados históricos corrigidos; nenhum segredo real localizado.	2 corrigidos
5	XSS Nenhum sink da aplicação; conteúdo dinâmico é texto React.	Sem achado

Tabela detalhada — arquivo por arquivo

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO VERIFICADA
MÉDIA	env.ts:46 env.test.ts:26-43	SEC-01 • CORRIGIDO. NODE_ENV não possui default; ausência e valor inválido são rejeitados por regressão.
MÉDIA	auth.ts:10-13,22-27 integration.test.ts:103-107	O modo de produção continua emitindo __Host-extraok_session com Secure, confirmado na integração real.
BAIXA	prisma.config.ts:3,11 prisma-config.test.ts:49-78	SEC-02 • CORRIGIDO. O config usa env("DATABASE_URL") e falha sem valor ou com valor vazio.
BAIXA	prisma.studio.config.ts:3-5 prisma.dev.config.ts:1-5	Studio exige URL e preserva max=1; o bootstrap local ganhou config sem datasource nem credencial.

23 handlers revisados	5 commits alcançáveis	2 achados corrigidos	0 segredos reais encontrados
--------------------------	--------------------------	-------------------------	---------------------------------

04 • ACHADO SEC-01

Corrigido — modo de execução agora falha fechado

MÉDIA

Corrigido

SEC-01 • Configuração / cookie de sessão

Na auditoria-base, a omissão de `NODE_ENV` ativava silenciosamente o perfil de desenvolvimento. A remediação removeu esse default e tornou o modo de execução obrigatório.

Antes e depois

```
// AUDITORIA-BASE - apps/api/src/config/env.ts:46-48
NODE_ENV: z
  .enum(["development", "test", "production"])
  .default("development"),

// REMEDIAÇÃO - apps/api/src/config/env.ts:46
NODE_ENV: z.enum(["development", "test",
"production"]),

// REGRESSÃO - apps/api/tests/env.test.ts:26-43
assert.throws(() =>
  loadEnv({ ...requiredEnvironment, NODE_ENV:
undefined })
);
assert.throws(() =>
  loadEnv({ ...requiredEnvironment, NODE_ENV:
"staging" })
);
```

Verificação automática: `env.test.ts:13-43` cobre ausência, valor inválido, `development` e `test`; `env.test.ts:70-98` cobre as restrições de `production`.

Risco original

Um deploy alternativo que omitisse `NODE_ENV` podia iniciar com cookie sem `Secure/___Host-`, aceitar origem HTTP remota e permitir o pepper público de desenvolvimento.

Correção aplicada

- `NODE_ENV` agora aceita apenas um modo informado explicitamente.
- `.env.example`, `Dockerfile` e `Compose` já definem seus modos esperados.
- A documentação passou a registrar a variável como obrigatória.

Resultado validado

A integração em modo de produção confirmou `___Host-extraok_session` e `Secure` (`integration.test.ts:37-48,103-107`). O fluxo real passou 1/1 e removeu os dados temporários ao final.

Status

Corrigido. A configuração inválida é recusada antes da criação do cliente Prisma e do servidor Fastify.

05 • ACHADO SEC-02

Corrigido — configs Prisma exigem DATABASE_URL

BAIXA

Corrigido

SEC-02 • CWE-798 / configuração local

Os fallbacks com credencial previsível foram removidos. Migration e Studio agora encerram o carregamento com erro claro quando DATABASE_URL está ausente ou vazia.

Antes e depois

```
// AUDITORIA-BASE - apps/api/prisma.config.ts:10-14
datasource: {
  url:
    process.env.DATABASE_URL ??
    "postgresql://extraok:extraok@127.0.0.1:5432/extraok?
schema=public",
},

// REMEDIAÇÃO - prisma.config.ts:3,11
import { defineConfig, env } from "prisma/config";
datasource: { url: env("DATABASE_URL") },

// REMEDIAÇÃO - prisma.studio.config.ts:3-5
import { defineConfig, env } from "prisma/config";
const databaseUrl = new URL(env("DATABASE_URL"));
```

Verificação automática: `prisma-config.test.ts:49-78` importa os dois configs em subprocessos isolados e cobre variável ausente, vazia e URL válida sem abrir conexão.

Risco original

Sem a variável, um comando podia operar silenciosamente contra um PostgreSQL local que aceitasse a combinação pública, causando leitura ou alteração no banco errado.

Correção aplicada

- `env("DATABASE_URL")` centraliza a leitura fail-closed nos dois configs.
- Studio mantém `max=1` apenas em sua URL.
- `prisma.dev.config.ts` preserva `create/start/stop` sem `datasource` nem fallback.

Controles preservados

Compose continua exigindo a URL por interpolação. O único placeholder restante está no estágio isolado de geração do Dockerfile e não é propagado para migration ou runtime.

Status

Corrigido. As seis regressões específicas e o build foram aprovados; a busca nos configs operacionais não encontrou o fallback removido.

06 • PONTOS FORTES

Controles confirmados com evidência no código

Isolamento consistente**01**

Cientes: `routes.ts:32-46, 58-60, 88-97, 108-127`.
Jobs: `routes.ts:55-80, 99-111, 143-159, 173-200`.
Extras: `routes.ts:23-33, 44-57, 69-98, 108-127`.

Dashboard escopado**02**

`dashboard/routes.ts:57-60, 73-113` aplica `ownerFilter` às agregações/listas e `ownerId` à lista de jobs.

Capability pública restrita**03**

Token aleatório de 256 bits e somente hash persistido (`tokens.ts:3-14`); decisão cruza `extraId` e `jobId` (`approvals/routes.ts:148-191`).

Sessão endurecida**04**

Cookie `HttpOnly`, `SameSite=Strict` e `Secure` em produção (`auth.ts:16-38`); expiração absoluta, ociosidade e revogação verificadas em `auth.ts:61-99`.

Sem sink XSS da aplicação**05**

Nenhum raw HTML, eval, markdown ou template HTML no backend. Dados são children React; redirects validam origem/allowlist (`Login-page.tsx:16-39`).

Bundle sem segredos**06**

`DATABASE_URL` e `PASSWORD_PEPPER` locais não aparecem no build web. A única variável pública é `VITE_API_URL` (`http-client.ts:3-4`).

Logs e links protegidos**07**

Token/query redigidos em `app.ts:37-56`; páginas e API públicas usam `no-store/no-referrer/noindex`; Nginx desliga access log nas URLs com token.

Deploy com fail-fast**08**

`NODE_ENV` é obrigatório no schema; configs Prisma exigem `DATABASE_URL`; Compose exige password, URL, origin e pepper. Containers de app seguem read-only e sem capabilities.

Também correto: updates/deletes repetem o filtro de dono na própria mutação; respostas 404 não distinguem objeto ausente de objeto alheio; IDs de dono não são aceitos nos schemas de entrada.

07 • COBERTURA DE ROTAS (1/2)

Infraestrutura, autenticação e clientes

HANDLER	GATE	ESCOPO VERIFICADO	EVIDÊNCIA
GET /health	Pública	Somente liveness; não consulta dados	app.ts:70
GET /ready	Pública	Consulta estática SELECT 1	app.ts:71-79
POST /auth/register	Pública	Cria o próprio usuário e sessão	auth/routes.ts:61-113
POST /auth/login	Pública	Credencial validada; erro genérico	auth/routes.ts:115-149
GET /auth/session	Sessão	requireAuth	auth/routes.ts:151-159
POST /auth/logout	Cookie	Revoga apenas o hash do token recebido	auth/routes.ts:161-179
GET /clients	Sessão	ownerId = usuário	clients/routes.ts:25-50
GET /clients/:id	Sessão	id + ownerId	clients/routes.ts:52-65
POST /clients	Sessão	ownerId definido pelo servidor	clients/routes.ts:67-79
PATCH /clients/:id	Sessão	update e reload com id + ownerId	clients/routes.ts:81-100
DELETE /clients/:id	Sessão	precheck, count e delete com dono	clients/routes.ts:102-130

Leitura da tabela

Sessão significa `preHandler: requireAuth`. **Cookie** no logout significa que a operação é idempotente e só consegue revogar o próprio token apresentado. Nenhuma linha depende do `ProtectedRoute` do navegador para proteger dados.

08 • COBERTURA DE ROTAS (2/2)

Atendimentos, extras, dashboard e aprovações

HANDLER	GATE	ESCOPO VERIFICADO	EVIDÊNCIA
GET /jobs	Sessão	ownerId em lista e busca relacionada	jobs/routes.ts:49-91
GET /jobs/:id	Sessão	id + ownerId antes dos includes	jobs/routes.ts:93-135
POST /jobs	Sessão	cliente validado + dono do servidor	jobs/routes.ts:137-164
PATCH /jobs/:id/status	Sessão	dono + transição + update condicional	jobs/routes.ts:166-204
GET /jobs/jobId/extras	Sessão	job.ownerId	extras/routes.ts:17-35
POST /jobs/jobId/extras	Sessão	jobId validado contra o dono	extras/routes.ts:37-60
PATCH /extras/:id	Sessão	job.ownerId + status pending	extras/routes.ts:62-100
DELETE /extras/:id	Sessão	job.ownerId + status pending	extras/routes.ts:102-137
GET /dashboard/summary	Sessão	ownerFilter em todas as 7 queries	dashboard/routes.ts:55-149
POST /jobs/jobId/approval-links	Sessão	jobId + ownerId antes da rotação	approvals/routes.ts:80-120
GET /public/approvals/token	Capability	hash + link ativo/expiração	approvals/routes.ts:20-74,122-132
POST /public/.../:extraId/decision	Capability	extraId + jobId derivado do token	approvals/routes.ts:134-198

Conclusão IDOR/tenant

Todos os endpoints que recebem identificadores privados combinam o ID com o dono autenticado, diretamente ou pela relação `job.ownerId`. No fluxo público, o token seleciona o job e o update exige o mesmo `jobId`; um token de um atendimento não opera um extra de outro.

09 • STATUS E PRÓXIMOS PASSOS

Remediação concluída e reforços futuros

P1**Concluído — modo de execução fail-closed**

Default de `NODE_ENV` removido; modos explícitos e restrições de produção cobertos por testes. SEC-01 encerrado.

Concluído em 02 de setembro de 2026 • testes focados e integração aprovados

P2**Concluído — configs Prisma sem credenciais padrão**

`DATABASE_URL` obrigatória em migration/Studio; bootstrap local isolado e regressões para ambos os configs. SEC-02 encerrado.

Concluído em 02 de setembro de 2026 • 6/6 regressões específicas aprovadas

P3**Automatizar a matriz negativa de isolamento**

Adicionar testes cross-tenant para cada listagem, detalhe, mutação, agregação e para `extraId` de outro job no fluxo público. O código atual está correto; isso evita regressões.

Hardening, fora da contagem de achados

P4**Adicionar varredura de segredos no CI futuro**

Quando CI for criado, verificar source, histórico e artefatos; bloquear URLs com credencial e chaves de alta confiança.

Hardening, fora da contagem de achados

P5**Reforçar invariantes no banco**

Considerar constraint composta/repository tenant-scoped para impedir que escritores futuros associem Job de um dono a Client de outro. Hoje não há caminho explorável pela API.

Defesa em profundidade, fora da contagem de achados

Limites da auditoria e da revalidação

A auditoria-base foi estática; a remediação teve regressões, integração local, typecheck, lint e build. O Compose não foi iniciado porque o daemon Docker local estava indisponível; Dockerfile/Compose foram reinspecionados. O trabalho não substitui pentest publicado, análise de CVEs, SAST ou testes de restauração/observabilidade.

10 • REGISTRO DE REMEDIAÇÃO (1/2)

SEC-01 — corrigido e validado

```
--- REGISTRO DE REMEDIAÇÃO SEC-01 ---
# [Segurança][Corrigido] Tornar a inicialização fail-closed para NODE_ENV

**Status:** corrigido e validado em 02 de setembro de 2026
**Severidade original:** média
**Auditoria-base:** `bb164eb957eaf688a4f7a5322750c6b49e5e7df8`
**Remediação:** incluída no commit que contém este relatório

## Problema original
O schema assumia `development` quando `NODE_ENV` era omitido. Isso desviava das validações de produção e permitia cookie sem `Secure`/`__Host-` em um deploy alternativo mal configurado.

## Correção aplicada
- Removido o default de `NODE_ENV` em `apps/api/src/config/env.ts:46`.
- `development`, `test` ou `production` agora precisam ser explícitos.
- A documentação da API passou a listar `NODE_ENV` como obrigatório.
- Dockerfile e Compose continuam declarando `production` explicitamente.

## Evidência atual
```ts
NODE_ENV: z.enum(["development", "test", "production"])
```

## Validação
- Regressões focadas: 12/12 aprovadas.
- Suíte padrão: 15 aprovadas; a integração foi executada separadamente.
- Integração real: 1/1 aprovada, confirmando `__Host-extraok_session` e `Secure`.
- `typecheck`, `lint`, `build` e `git diff --check`: aprovados.

## Critérios de aceite
- [x] Ausência e valor inválido são rejeitados.
- [x] `development`, `test` e `production` são cobertos.
- [x] Produção rejeita origem HTTP remota e pepper local conhecido.
- [x] A integração confirma cookie de produção endurecido.
- [x] Dockerfile/Compose preservam `NODE_ENV=production` por inspeção.

**Limite:** o Compose não foi iniciado nesta revalidação; o daemon Docker local estava indisponível.
--- FIM DO REGISTRO SEC-01 ---
```

11 • REGISTRO DE REMEDIAÇÃO (2/2)

SEC-02 — corrigido e validado

```
--- REGISTRO DE REMEDIAÇÃO SEC-02 ---
# [Segurança][Corrigido] Remover credenciais padrão dos configs Prisma

**Status:** corrigido e validado em 02 de setembro de 2026
**Severidade original:** baixa
**Auditoria-base:** `bb164eb957eaf688a4f7a5322750c6b49e5e7df8`
**Remediação:** incluída no commit que contém este relatório

## Problema original
Migration e Studio usavam uma URL PostgreSQL previsível quando `DATABASE_URL` estava ausente, com risco de operar silenciosamente no banco errado.

## Correção aplicada
- Os dois configs usam `env("DATABASE_URL")` de `prisma/config`.
- Variável ausente ou vazia encerra o carregamento com erro claro.
- Studio preserva `max=1` sem alterar o pool da API.
- `prisma.dev.config.ts` mantém o bootstrap local sem datasource/fallback.

## Evidência atual
```ts
datasource: { url: env("DATABASE_URL") }
const databaseUrl = new URL(env("DATABASE_URL"))
```

## Validação
- Seis testes cobrem ambos os configs com variável ausente, vazia e URL válida.
- URL válida é carregada em subprocesso isolado sem conexão com banco.
- Busca nos fontes/configs confirmou a remoção de `extraok:extraok`.
- O placeholder do Docker permanece restrito ao estágio de geração.

## Critérios de aceite
- [x] Nenhum config operacional contém credencial de fallback.
- [x] Migration e Studio falham sem `DATABASE_URL`.
- [x] Valor vazio também é rejeitado.
- [x] Os dois configs têm regressões automatizadas.
- [x] O Prisma Client e o build do workspace foram gerados com sucesso.
- [x] O bootstrap `prisma dev` funciona sem reintroduzir fallback.

**Limite:** a imagem Docker não foi reconstruída; seu placeholder isolado foi verificado por inspeção.
--- FIM DO REGISTRO SEC-02 ---
```